Grupo 18
Nome: Arthur Trottmann Ramos NUSP: 14681052
Nome: Henrique Drago NUSP: 14675441
Nome: Henrique Yukio Sekido NUSP: 14614564
Nome: João Vitor NUSP: 13748102

Github: https://github.com/ArthurTRamos/Projeto_Final-BD

Descrição do Problema

Uma instituição de ensino (escola) necessita de um sistema online para gerenciar suas atividades acadêmicas e administrativas, facilitando a interação entre alunos, professores e a administração. A escola oferece diversos cursos e disciplinas em diferentes níveis de ensino.

Este trabalho visa criar uma base de dados para armazenar as informações envolvidas nas atividades da escola. Dentre essas informações está a descrição dos **Cursos** oferecidos, que deve estar identificada por pelo menos um nome, um código único e o departamento ao qual pertence. Cada curso é classificado por um nível de ensino (ex: fundamental, médio, técnico, graduação) e sua descrição deve incluir a carga horária total, o número de vagas, e pode indicar ainda a ementa, os pré-requisitos (outros cursos ou disciplinas) e a sala de aula padrão onde é ministrado.

Cada **Disciplina** que compõe um curso deve indicar quantas aulas semanais ela possui e o material didático básico recomendado. Cada curso pode ter diversas **Regras** (ex: frequência mínima obrigatória, critérios de aprovação) e diversas **Necessidades de Infraestrutura** (ex: laboratório de informática, projetor, lousa digital).

A escola contempla três tipos principais de **Usuários**: o **Professor**, que é quem ministra as disciplinas; o **Aluno**, que se matricula nas disciplinas; e o **Funcionário Administrativo**, que gerencia as operações. Todos os usuários têm um nome, sobrenome, data de nascimento, endereço, sexo, número de telefone, endereço de e-mail e uma senha para acesso ao sistema. Assuma que não existem dois usuários com o mesmo nome, sobrenome e número de telefone.

Cada **Matrícula** é feita por um aluno em uma ou mais disciplinas de um curso para um determinado período letivo (ex: semestre ou ano). Cada matrícula deve incluir também uma data da matrícula (quando foi efetivada), o status da matrícula (ex: ativa, trancada, concluída, reprovada) e as notas obtidas pelo aluno naquela disciplina. Pode incluir também informações sobre bolsas de estudo ou descontos aplicados. Note-se que cada matrícula em uma disciplina se refere a um único aluno e a uma oferta de disciplina específica. Uma matrícula pode ser confirmada ou

cancelada até uma data limite antes do início do período letivo, e se confirmada, pode envolver o pagamento de taxas pelo aluno.

Cada disciplina pode ter muitas matrículas, mas o número de alunos matriculados não pode exceder a capacidade da turma ou da sala de aula designada. Cada disciplina é de responsabilidade de um ou mais professores, e cada professor pode ter informações sobre sua área de especialização e titulação. A escola também mantém um registro de **Departamentos Acadêmicos**, onde cada departamento possui um nome, um código único e um professor designado como chefe do departamento. Um departamento só pode ter um chefe.

Além disso, cada curso, disciplina, aluno e professor estão associados a uma **Unidade da Escola** (campus ou localidade específica), que corresponde à cidade, estado e país, e possivelmente a um prédio ou bloco específico dentro da unidade.

A comunicação no sistema é feita por meio de **Avisos** e **Mensagens**. Um professor pode enviar 0 ou mais mensagens para seus alunos (individualmente ou para a turma) e vice-versa. A administração também pode enviar avisos gerais. Cada mensagem é identificada por quem a enviou, o destinatário (ou grupo de destinatários) e o timestamp de sua criação, e deve incluir o texto da mensagem.

Um aluno pode **Avaliar** zero ou mais disciplinas e/ou professores ao final de um período letivo. Cada avaliação pode incluir um comentário em texto, e classificações para didática do professor, material de apoio, relevância do conteúdo e infraestrutura da sala.

1. Tarefas

1.1 Identificação dos Elementos

Descreva, em forma de texto corrido, todos os conjuntos entidades (CE) identificados na problemática e seus respectivos atributos. Em seguida, identifique todos os conjuntos relacionamentos (CRs) entre as entidades, sempre explicando a lógica por trás da escolha da multiplicidade e participação de cada entidade na relação. Identifique também os conjuntos entidades fracos (CEF, se houver), juntamente com CRT que o define como fraco, atributos multivalorados, atributos compostos, atributos de CRs.

a) A princípio, listamos os conjuntos de entidades identificados na problemática.

Curso

- Nome
- Código Único (Chave)
- Nível de Ensino
- Carga Horária Total
- N° de Vagas

- Ementa

Conjunto de Entidades “Curso” criado para conter as informações de diferentes cursos criados pela instituição de ensino.

Sala

- ID (Chave)
- Capacidade

Conjunto de Entidades “Sala” criado devido a possibilidade de uma turma ter uma sala diferente da padrão do curso (especulação) e da existência de uma limitação de capacidade para a sala independente da limitação da turma (inferido no texto). Recebeu um ID para chaveamento.

Disciplina

- ID (Chave)
- Nome
- Aulas Semanais
- Material Didático Recomendado (multivalorado)

Conjunto de Entidades “Disciplina” criado para conter as informações de diferentes disciplinas criadas pela instituição de ensino. Recebeu um ID para chaveamento, visto que o atributo “Nome” não é garantido como único.

Regra

- ID (Chave)
- Descrição

Conjunto de Entidades “Regra” criado para conter as informações de diferentes regras criadas pela instituição de ensino, devido a possibilidade de outras entidades estarem relacionadas a diversas regras possivelmente sobrepostas. Recebeu um ID para facilitar o chaveamento, visto que o atributo “Descrição” não tem garantia de unicidade e ser possivelmente extenso demais, dificultando sua utilização como chave.

Necessidade de Infraestrutura

- ID (Chave)
- Descrição

Conjunto de Entidades “Necessidade de Infraestrutura” criado para conter as informações de diferentes necessidades de infraestrutura criadas pela instituição de ensino, devido a possibilidade de outras entidades estarem relacionadas a diversas necessidades de infraestrutura possivelmente sobrepostas. Recebeu um ID para facilitar o chaveamento, visto que o atributo “Descrição” não tem garantia de unicidade e ser possivelmente extenso demais, dificultando sua utilização como chave.

Usuário [Generalização]

- ID (Chave)
- Nome (Composto: Nome, Sobrenome)
- Data de Nascimento
- Endereço (Composto: País, Estado, Cidade, Bairro, Rua, Número, CEP)
- Sexo
- Telefone
- Email
- Senha
- Tipo (Atributo de Especialização)
- **Subclasses:**
    - **Aluno**
    - **Professor**
        - Área de Especialização (multivalorado)
        - Titulação (multivalorado)
    - **Funcionário Administrativo**

Obs: não existem dois usuários com o mesmo nome, sobrenome e número de telefone.

A generalização Usuário foi utilizada porque as entidades Aluno, Professor e Funcionário Administrativo compartilham um conjunto extenso de atributos em comum. Todos são "pessoas" no sistema e possuem dados como Nome, Sobrenome, Data de Nascimento, Endereço, Email e Senha. Criar uma superclasse Usuário para abrigar esses atributos evita a redundância e a inconsistência de dados que ocorreriam se essas colunas fossem repetidas em três tabelas separadas.

A separação entre subclasses se faz necessária devido às diferentes relações delas e os atributos adicionais de professor.

Matrícula

- ID (Chave)
- Data de Efetivação
- Status
- Bolsa (opcional)
- Desconto (opcional)
- Taxa de Cancelamento (opcional)
- Confirmação

Conjunto de Entidades “Matrícula” criado para conter as informações de diferentes matrículas registradas.

Turma (ou Oferta de Disciplina) [Fraco. CRT: Oferta (de Disciplina)]

- Período Letivo (Chave)
- Capacidade

Conjunto de Entidades “Turma” criado para conter as informações de diferentes ofertas de disciplinas oferecidas a cada período letivo. Julgado necessário devido a possibilidade de matrículas se referirem a diversas disciplinas ao mesmo tempo, se referindo a períodos letivos específicos.

Departamento Acadêmico

- Nome
- Código Único (Chave)

Conjunto de Entidades “Departamento Acadêmico” criado para conter as informações de diferentes departamentos acadêmicos criados pela instituição de ensino.

Unidade da Escola

- ID (Chave)
- Cidade
- Estado
- País
- Prédio/Bloco (opcional)

Conjunto de Entidades “Unidade da Escola” criado para conter as informações de diferentes unidades gerenciadas pela instituição de ensino. Adicionado ID para chaveamento, visto que as outras informações não são necessariamente únicas.

Comunicação [Generalização]

- ID (Chave)
- Timestamp
- Conteúdo
- Tipo (Atributo de Especialização)
- **Subclasses:**
    - **Avisos**
    - **Mensagens**

A generalização Comunicação foi adotada para agrupar as entidades Aviso e Mensagem, que compartilham atributos fundamentais que definem um ato comunicativo no sistema: um identificador único (ID), um registro de quando ocorreu (Timestamp) e o próprio conteúdo. Agrupá-los na superclasse Comunicação estabelece um conceito unificado e evita a repetição desses atributos.

A divisão em subclasses se faz necessária devido aos diferentes relacionamentos e naturezas dessas classes.

b) Em seguida, listamos os conjuntos de relacionamento e suas respectivas multiplicidades.

Conjuntos de Relacionamentos

Curso (1, 1) – (0, N) Departamento [Pertence/Contém]

- Um curso pertence a um departamento e um departamento tem nenhum ou vários cursos.

Disciplina (0, N) – (0, N) Regra [Regulada/Regula]

- Uma disciplina é regulada por nenhuma ou várias regras, e uma regra regula uma ou várias disciplinas.

Disciplina (0, N) – (0, N) Necessidade de Infraestrutura [Requer/Requerida]

- Uma disciplina requer nenhuma ou várias infraestruturas, e uma infraestrutura é requerida por uma ou várias disciplinas.

Professor (0, N) – (1, N) Turma [Ministra/Ministrada]

- Um professor ministra nenhuma ou várias turmas, uma turma é ministrada por um ou vários professores.

Professor (0, N) – (1, 1) Departamento [Chefia/Chefiado]

- Um professor chefia nenhum ou vários departamentos (não foi especificado, então assumiu-se assim), um departamento é chefiado por um professor.

Disciplina (0, N) – (1, N) Curso [Compõe/Composto]

- Uma disciplina compõe nenhum ou vários cursos, um curso é composto por uma ou várias disciplinas.

Usuário (0, N) – Mensagem (1, 1) [Envia/Enviada]

- Um usuário envia nenhuma ou várias mensagens, uma mensagem é enviada por um usuário.

Usuário (0, N) – Mensagem (0, N) [Recebe/Recebida]

- Um usuário recebe nenhuma ou várias mensagens, uma mensagem é recebida por um ou vários usuários.

Turma (0, N) – Mensagem (0, 1) [Recebe/Recebida]

- Uma turma recebe nenhuma ou várias mensagens, uma mensagem é recebida por nenhuma ou uma turma.

Funcionário Administrativo (0, N) – (1, 1) Aviso [Faz/Feito]

- Um funcionário administrativo faz nenhum ou vários avisos, um aviso é feito por um funcionário administrativo.

Usuário (0, N) – (1, N) Aviso [Avisado/Avisa]

- Um usuário é avisado por nenhum ou vários avisos, um aviso avisa um ou vários usuários.
- Obs: um aviso é feito sempre para o sistema inteiro, por isso foi escolhido usuário para a relação ao invés de destinatário. E sempre será ao menos um, pois alguém tem que enviar o aviso, logo existe ao menos um usuário de destino.

Aluno (0, N) – (0, N) Turma [Avalia/Avaliado] [Abstração]

- Um aluno avalia nenhuma ou várias turmas, uma turma é avaliada por nenhum ou vários alunos.
- Atributos: ID (Chave), Comentário, Classificação de Didática, Classificação de Material de Apoio, Classificação de Conteúdo, Classificação de Infraestrutura.
- Justificativa: A relação Avaliação representa uma Agregação, conforme a necessidade de identificar cada instância do relacionamento com uma chave própria (ID_Avaliacao). A tabela resultante contém essa chave primária artificial, além das chaves estrangeiras (ID_Aluno, ID_Disciplina, Periodo_Letivo) que referenciam as entidades participantes do relacionamento agregado.

Unidade da Escola (0, N) – (1, 1) Aluno [Localiza/Localizado]

- Uma unidade da escola localiza um aluno, um aluno é localizado em uma unidade da escola.

Unidade da Escola (0, N) – (1, 1) Professor [Localiza/Localizado]

- Uma unidade da escola localiza um professor, um professor é localizado em uma unidade da escola.

Unidade da Escola (0, N) – (1, 1) Disciplina [Localiza/Localizado]

- Uma unidade da escola localiza uma disciplina, uma disciplina é localizada em uma unidade da escola.

Unidade da Escola (0, N) – (1, 1) Curso [Localiza/Localizado]

- Uma unidade da escola localiza um curso, um curso é localizado em uma unidade da escola.

Curso (0, N) – (0, N) Curso [Pré-Requisita]

- Um curso tem nenhum ou vários cursos de pré-requisito, um curso é pré-requisito de nenhum ou vários cursos.

Curso (0, N) – (0, N) Disciplina [Pré-Requisita]

- Um curso tem nenhuma ou várias disciplinas de pré-requisito, uma disciplina é pré-requisito de nenhum ou vários cursos.

Curso (1, 1) – (0, N) Sala [Padrão]

- Um curso tem uma sala de aula padrão, uma sala de aula pode ser padrão para nenhum ou vários cursos.

Matrícula (1, 1) – (1, N) Aluno [Corresponde/Tem]

- Cada matrícula corresponde a um aluno, cada aluno tem uma ou várias matrículas.

Matrícula (1, N) – (0, N) Turma [Corresponde/Tem]

- Atributos: Nota
- Cada matrícula corresponde a uma turma, cada turma tem nenhuma ou várias matrículas.

Turma (1, 1) – (0, N) Disciplina [Ofertada/Oferta]

- Cada turma é oferta de uma disciplina, cada disciplina oferta nenhuma ou várias turmas.

Turma (1, 1) – (0, N) Sala [Designada]

- Cada turma é designada a uma sala, cada sala é designada a nenhuma ou várias turmas.

1.2 Diagrama Entidade-Relacionamento

Faça a modelagem E-R da base de dados descrita para o sistema de gestão escolar, utilizando o Modelo Entidade-Relacionamento Estendido (ME-RX). Seu diagrama deve representar explicitamente pelo menos uma abstração de Generalização/Especialização e uma de Agregação, caso se apliquem naturalmente ao problema.

O diagrama produzido neste exercício encontra-se ao final deste documento.

1.3 Mapeamento

Faça o mapeamento completo do Modelo Entidade-Relacionamento Estendido (ME-RX), obtido no Exercício 2, para o Modelo Relacional. Sua resposta deve incluir a definição de todas as relações, seus atributos com os respectivos domínios, as chaves primárias, chaves candidatas e chaves estrangeiras. Justifique as decisões de mapeamento para construções mais complexas (ex: generalizações/especializações, atributos multivalorados, diferentes multiplicidades).

Para cada relação abaixo, utilizaremos as seguintes sintaxes para representar chaves primárias e estrangeiras

PK = Chave Primária

FK = Chave Estrangeira

Observamos ainda que foi utilizado o procedimento padrão G1 para mapeamento das abstrações de generalização, conforme os slides da disciplina.

Curso = {Codigo_Unico (PK), Nome, Nivel_Ensino, Carga_Horaria_Total, Numero_Vagas, Ementa, Codigo_Unico_Departamento (de Departamento) (FK), ID_Unidade (de Unidade da Escola) (FK), ID_SalaPadrao (de Sala) (FK)}

Sala = {ID (PK), Capacidade}

Disciplina = {ID (PK), Nome, Aulas_Semanais, Material_Didatico_Recomendado, ID_Unidade (de Unidade da Escola) (FK)}

Regra = {ID (PK), Descricao}

Necessidade_Infraestrutura = {ID (PK), Descricao}

Usuario = {ID (PK), Nome, Sobrenome, Data_Nascimento, Pais, Estado, Cidade, Bairro, Rua, Numero, CEP, Sexo, Telefone, Email, Senha, Tipo}

- Chaves Secundárias: [Nome + Sobrenome + Telefone], [Email]

Aluno = {ID_Aluno (PK, FK de Usuario), ID_Unidade (de Unidade da Escola) (FK)}

Professor = {ID_Professor (PK, FK de Usuario), ID_Unidade (de Unidade da Escola) (FK)}

Area_Especializacao = {ID_Professor (de Professor) (PK, FK), Area (PK)}

Titulacao = {ID_Professor (de Professor) (PK, FK), Titulo (PK)}

Funcionario_Administrativo = {ID_Func_Admin (PK, FK de Usuario)}

Matricula = {ID_Matricula (PK), Data_Matricula, Status, ID_Aluno (de Aluno) (FK), Bolsa, Taxa, Desconto}

Turma = {ID_Disciplina (de Disciplina) (PK, FK), Periodo_Letivo (PK), Capacidade, ID_Sala (de Sala) (FK)}

Departamento_Academico = {Codigo_Unico (PK), Nome, ID_Professor_Chefe (de Professor) (FK)}

Unidade_Escola = {ID_Unidade (PK), Cidade, Estado, Pais, Predio_Bloco}

Comunicação = {ID_Comunicacao (PK), Data_Hora, Conteudo, Tipo}

Avisos = {ID_Aviso (PK, FK de Comunicacao), ID_Func_Admin (de Funcionario_Administrativo) (FK)}

Mensagens = {ID_Mensagem (PK, FK de Comunicacao), ID_Remetente (de Usuario) (FK), ID_Turma_Disciplina (de Turma) (FK), Periodo_Letivo_Turma (de Turma) (FK)}

Mensagem_Para_Usuario = {ID_Mensagem (de Mensagens) (PK, FK), ID_Usuario_Destinatario (de Usuario) (PK, FK)}

Avaliacao = {ID_Avaliacao (PK), ID_Aluno (de Aluno) (FK), ID_Disciplina (de Turma) (FK), Periodo_Letivo (de Turma) (FK), Comentario, Classificacao_Didatica, Classificacao_Material_Apoio, Classificacao_Conteudo, Classificacao_Infraestrutura}

- Chaves Secundárias: [ID_Aluno, ID_Disciplina, Periodo_Letivo]

Regra_Regula_Disciplina = {ID_Regra (de Regra) (PK, FK), ID_Disciplina (de Disciplina) (PK, FK)}

Disciplina_Requer_Infraestrutura = {ID_Disciplina (de Disciplina) (PK, FK), ID_Necessidade_Infraestrutura (de Necessidade_Infraestrutura) (PK, FK)}

Professor_Ministra_Turma = {ID_Professor (de Professor) (PK, FK), ID_Disciplina (de Turma) (PK, FK), Periodo_Letivo (de Turma) (PK, FK)}

Disciplina_Compoe_Curso = {ID_Disciplina (de Disciplina) (PK, FK), Codigo_Unico_Curso (de Curso) (PK, FK)}

Usuario_AvisadoPor_Aviso = {ID_Usuario (de Usuario) (PK, FK), ID_Aviso (de Avisos) (PK, FK)}

Curso_PreRequisita_Curso = {Codigo_Unico_Curso (de Curso) (PK, FK), Codigo_Unico_Curso_PreRequisito (de Curso) (PK, FK)}

Curso_PreRequisita_Disciplina = {Codigo_Unico_Curso (de Curso) (PK, FK), ID_Disciplina_PreRequisito (de Disciplina) (PK, FK)}

Matricula_Corresponde_Turma = {ID_Matricula (de Matricula) (PK, FK), ID_Disciplina (de Turma) (PK, FK), Periodo_Letivo (de Turma) (PK, FK), Nota_Final}

1.4 População das Relações

Popule as tabelas resultantes do mapeamento (Exercício 3) com dados criados manualmente ou de forma sintética. Cada tabela principal (ex: Aluno e Professor) deve conter no mínimo 20 registros, e as tabelas resultantes de CRs (ex: Matrícula) devem conter um volume de dados consistente que permita a execução das consultas futuras. Os dados devem ser entregues em formato de scripts SQL de inserção (.sql) junto à atividade.

As relações propostas pelo mapeamento foram populadas e o script SQL utilizado encontra-se na página do Github.

1.5 Normalização

Analise o esquema relacional obtido no Exercício 3 e justifique a forma normal (ex: 1FN, 2FN, 3FN, BCNF) em que cada tabela se encontra, logo após a modelagem. Depois, com base nas dependências funcionais identificadas, normalize toda a base até a BCNF (considerando as que ainda não se encontram nas FNs). Ao aplicar o processo de normalização, detalhe os passos seguidos e apresente o esquema relacional final, normalizado.

Para o processo de normalização, analisamos, relação a relação, se as condições impostas pelas formas normais requisitadas estavam presentes. Caso estas não estivessem presentes, aplicamos o processo de normalização necessário.

Relação Curso

- **1FN:** A relação possui atributos atômicos e monovalorados. Estamos considerando que a ementa será apenas uma string e que não será necessário acessar cada conteúdo presente nela.
- **2FN:** Temos que, pela chave primária ter apenas um atributo, então todos os atributos restantes dependem funcionalmente dela. Em outras palavras:

  Código Único –> Restante dos Atributos
- **3FN:** todos os atributos dependem diretamente da chave primária.
- **BCNF:** Não há chaves candidatas.

Relação Sala

- Está nas formas normais 1, 2, 3 e BCNF (conforme a explicação anterior).

Relação Disciplina

- **1FN:** A relação possui apenas atributos atômicos. Porém, possui o atributo Material_Didatico_Recomendado, o qual pode possuir vários materiais (strings). Logo, é um atributo multivalorado. Com isso, as relações resultantes da normalização são:

  Disciplina = {ID (PK), Nome, Aulas_Semanais, ID_Unidade (de Unidade da Escola) (FK)}

  Materiais_Didaticos_Recomendados = {ID_Disciplina (PK) (de Disciplina) (FK), Material_Didatico_Recomendado (PK)}

- **2FN:** Todos os atributos restantes de Disciplina dependem funcionalmente da chave primária ID. Ou seja:

  ID –> Atributos restantes
- **3FN:** Todos os atributos não-chave dependem diretamente de ID.
- **BCNF:** Não há chaves candidatas.

Relação Regra

- Está nas formas normais 1, 2, 3 e BCNF.

Relação Necessidade_Infraestrutura

- Está nas formas normais 1, 2, 3 e BCNF.

Relação Usuário

- **1FN:** Consideramos data_nascimento como atributo atômico (já que não há partes, como dia e mês, em outras relações do banco). Logo, a relação está na 1FN.
- **2FN:** Por ter apenas um atributo compondo a chave primária, a relação está na 2FN.
- **3FN:** A relação não está na 3F. Na dependência funcional

ID –> CEP –> País, Estado, Cidade, Bairro, Rua

Temos uma dependência transitiva. Logo, temos as seguintes relações resultantes da normalização:

Usuário = {ID (PK), Nome, Sobrenome, Telefone, Data_Nascimento, Numero, CEP, Sexo, Email, Senha, Tipo}

Chaves Secundárias: [Nome + Sobrenome + Telefone], [Email]

Endereço_Usuario = {CEP (PK) (de Usuário) (FK), País, Estado, Cidade, Bairro, Rua}

Além disso, temos a dependência:

ID –> Email –> Senha

Logo, temos as seguintes relações resultantes:

Usuário = {ID (PK), Nome, Sobrenome, Telefone, Data_Nascimento, Numero, CEP, Sexo, Email, Tipo}

Chaves Secundárias: [Nome + Sobrenome + Telefone], [Email]

Endereco_Usuario = {CEP (PK) (de Usuário) (FK), Pais, Estado, Cidade, Bairro, Rua}

Login_Usuario = {Email (PK) (de Usuário) (FK), Senha}

- **BCNF:** As relações já estão na BCNF.

Relação Aluno, Professor, Funcionário Administrativo

- Estão nas formas normais 1, 2, 3 e BCNF.

Relações Area_Especializacao e Titulação

- Estão nas formas normais 1 (considerando o atributo Area como monovalorado), 2, 3 e BCNF.

Relação Matrícula

- **1FN:** Não há atributos não atômicos ou multivalorados.
- **2FN:** Como há apenas uma chave primária, a relação está na 2FN.
- **3FN:** Todos os atributos fora da chave primária têm dependência não-transitiva com ela. Note que, pelo fato de ID_Aluno poder se repetir, não há a dependência ID_Aluno –> Bolsa, Taxa, Desconto.
- **BCNF:** Não há chaves candidatas.

Relação Turma

- **1FN:** Considerando o periodo_letivo como atômico, a relação está na 1FN.
- **2FN:** Os atributos não-chave da relação dependem inteiramente da chave primária.
- **3FN:** Não há dependências transitivas.
- **BCNF:** Não há chaves candidatas.

Relação Departamento_Academico

- **1FN:** Todos os atributos são atômicos e monovalorados.
- **2FN:** Como só há um atributo compondo a chave primária, a relação está na 2FN.
- **3FN:** Note que não temos a dependência Nome –> ID_Professor_Cheve, pois podemos ter departamentos de mesmo nome (em unidades diferentes, por exemplo), mas com diferentes professores chaves. Da mesma forma, não temos ID_Professor_Cheve –> Nome, pois um professor pode ser chefe de mais de um departamento.
- **BCNF:** Não há chaves candidatas.

Relação Unidade_Escola

- **1FN:** Como prédio e bloco não são utilizados separadamente em outras relações do banco, então eles são mantidos como atômicos
- **2FN:** Como só há um atributo compondo a chave primária, a relação está na 2FN.
- **3FN:** Não há dependências transitivas. Logo, a relação está na 3FN.
- **BCNF:** Não há chaves candidatas.

Relação Comunicação

- Considerando o Data_Hora como atributo atômico, a relação está nas formas normais descritas.

Relação Avisos

- A relação está nas formas normais descritas.

Relação Mensagens

- **1FN:** Considerando período letivo como atômico, a relação está na 1FN.

- **2FN:** Possui apenas um atributo compondo a chave primária.
- **3FN:** Todos os atributos dependem funcionalmente de maneira direta à chave primária.
- **BCNF:** Não há chaves candidatas.

Relação Mensagem_Para_Usuário
- Está nas formas normais 1, 2, 3 e BCNF.

Relação Avaliação
- Está nas formas normais descritas.

Relações Regra_Regula_Disciplina, Disciplina_Requer_Infraestrutura, Disciplina_Compoe_Curso
- Estão nas formas normais 1, 2, 3 e BCNF.

Relações Usuario_AvisadorPor_Aviso, Curso_Pre_Requisita_Curso e Curso_PreRequisita_Disciplina
- Estão nas formas normais 1, 2, 3 e BCNF.

Relação Professor_Ministra_Turma
- A relação está nas formas normais descritas.

Relação Matricula_Corresponde_Turma
- A relação está nas formas normais descritas.

A partir do processo de normalização descrito, temos as seguintes relações do mapeamento final:

Curso = {Codigo_Unico (PK), Nome, Nivel_Ensino, Carga_Horaria_Total, Numero_Vagas, Ementa, Codigo_Unico_Departamento (de Departamento) (FK), ID_Unidade (de Unidade da Escola) (FK), ID_SalaPadrao (de Sala) (FK)}

Sala = {ID (PK), Capacidade}

Disciplina = {ID (PK), Nome, Aulas_Semanais, ID_Unidade (de Unidade da Escola) (FK)}

Materiais_Didaticos_Recomendados = {ID_Disciplina (PK) (de Disciplina) (FK), Material_Didatico_Recomendado (PK)}

Regra = {ID (PK), Descricao}

Necessidade_Infraestrutura = {ID (PK), Descricao}

Usuário = {ID (PK), Nome, Sobrenome, Data_Nascimento, Numero, CEP (de Endereco_Usuario)(FK), Sexo, Email, Tipo}

- Chaves Secundárias: [Nome + Sobrenome + Telefone], [Email]

Telefones_Usuário = {ID_Usuário (PK) (de Usuário) (FK), Telefone (PK)}

Endereco_Usuario = {CEP (PK), Pais, Estado, Cidade, Bairro, Rua}

Login_Usuario = {Email (PK) (de Usuário) (FK), Senha}

Aluno = {ID_Aluno (PK, FK de Usuario), ID_Unidade (de Unidade da Escola) (FK)}

Professor = {ID_Professor (PK, FK de Usuario), ID_Unidade (de Unidade da Escola) (FK)}

Area_Especializacao = {ID_Professor (de Professor) (PK, FK), Area (PK)}

Titulacao = {ID_Professor (de Professor) (PK, FK), Titulo (PK)}

Funcionario_Administrativo = {ID_Func_Admin (PK, FK de Usuario)}

Matricula = {ID_Matricula (PK), Data_Matricula, Status, ID_Aluno (de Aluno) (FK), Bolsa, Taxa, Desconto}

Turma = {ID_Disciplina (de Disciplina) (PK, FK), Periodo_Letivo (PK), Capacidade, ID_Sala (de Sala) (FK)}

Departamento_Academico = {Codigo_Unico (PK), Nome, ID_Professor_Chefe (de Professor) (FK)}

Unidade_Escola = {ID_Unidade (PK), Cidade, Estado, Pais, Predio_Bloco}

Comunicação = {ID_Comunicacao (PK), Data_Hora, Conteudo, Tipo}

Avisos = {ID_Aviso (PK, FK de Comunicacao), ID_Func_Admin (de Funcionario_Administrativo) (FK)}

Mensagens = {ID_Mensagem (PK, FK de Comunicacao), ID_Remetente (de Usuario) (FK), ID_Turma_Disciplina (de Turma) (FK), Periodo_Letivo_Turma (de Turma) (FK)}

Mensagem_Para_Usuario = {ID_Mensagem (de Mensagens) (PK, FK), ID_Usuario_Destinatario (de Usuario) (PK, FK)}

Avaliacao = {ID_Avaliacao (PK), ID_Aluno (de Aluno) (FK), ID_Disciplina (de Turma) (FK), Periodo_Letivo (de Turma) (FK), Comentario, Classificacao_Didatica, Classificacao_Material_Apoio, Classificacao_Conteudo, Classificacao_Infraestrutura}

- Chaves Secundárias: [ID_Aluno, ID_Disciplina, Periodo_Letivo]

Regra_Regula_Disciplina = {ID_Regra (de Regra) (PK, FK), ID_Disciplina (de Disciplina) (PK, FK)}

Disciplina_Requer_Infraestrutura = {ID_Disciplina (de Disciplina) (PK, FK), ID_Necessidade_Infraestrutura (de Necessidade_Infraestrutura) (PK, FK)}

Professor_Ministra_Turma = {ID_Professor (de Professor) (PK, FK), ID_Disciplina (de Turma) (PK, FK), Periodo_Letivo (de Turma) (PK, FK)}

Disciplina_Compoe_Curso = {ID_Disciplina (de Disciplina) (PK, FK), Codigo_Unico_Curso (de Curso) (PK, FK)}

Usuario_AvisadoPor_Aviso = {ID_Usuario (de Usuario) (PK, FK), ID_Aviso (de Avisos) (PK, FK)}

Curso_PreRequisita_Curso = {Codigo_Unico_Curso (de Curso) (PK, FK), Codigo_Unico_Curso_PreRequisito (de Curso) (PK, FK)}

Curso_PreRequisita_Disciplina = {Codigo_Unico_Curso (de Curso) (PK, FK), ID_Disciplina_PreRequisito (de Disciplina) (PK, FK)}

Matricula_Corresponde_Turma = {ID_Matricula (de Matricula) (PK, FK), ID_Disciplina (de Turma) (PK, FK), Periodo_Letivo (de Turma) (PK, FK), Nota_Final}

1.6 Consultas SQL

Neste exercício, criamos 3 tipos de scripts: criação de tabelas (create table), inserção de dados (insert into) e consultas (select). Todos os códigos podem ser encontrados no Github do projeto. Com relação aos comandos de consulta, construímos os 7 seguintes scripts:

1. Consultar as disciplinas em que cada aluno está matriculado e sua respectiva nota final.

```sql
-- Consulta o Aluno (Nome e Sobrenome), sua disciplina matriculada e sua respectiva nota final

SELECT
    U.Nome || ' ' || U.Sobrenome AS nome_completo,
    MCT.ID_Disciplina,
    MCT.Nota_Final AS nota_final
FROM
    Usuario U
JOIN
    Matricula M ON M.ID_Aluno = U.ID
JOIN
    Matricula_Corresponde_Turma MCT ON MCT.ID_Matricula = M.ID_Matricula
JOIN
    Turma T ON T.ID_Disciplina = MCT.ID_Disciplina AND T.Periodo_Letivo = MCT.Periodo_Letivo
WHERE
    U.Tipo = 'Aluno'
ORDER BY
    U.Nome, U.Sobrenome;
```

2. Consultar todos os alunos matriculados em uma disciplina em um determinado período.

```sql
-- Consulta todos os alunos matriculados em uma disciplina em um determinado período;

SELECT
    U.Nome || ' ' || U.Sobrenome AS nome_aluno,
    M.Data_Matricula AS data_matricula,
    D.Nome AS disciplina,
    MCT.Periodo_Letivo AS periodo_letivo
FROM
    Usuario U
JOIN
    Matricula M ON M.ID_Aluno = U.ID
JOIN
    Matricula_Corresponde_Turma MCT ON MCT.ID_Matricula = M.ID_Matricula
JOIN
    Disciplina D ON D.ID = MCT.ID_Disciplina

WHERE
    U.Tipo = 'Aluno' AND
    M.Status = 'Ativa'
ORDER BY
    D.Nome;
```

3. Consultar e listar os professores de um departamento com suas respectivas disciplinas.

```sql
-- Consultar e listar os professores de um departamento com suas respectivas disciplinas

SELECT DISTINCT
    U.Nome || ' ' || U.Sobrenome AS Nome_Professor,
    DA.Nome AS Departamento,
    D.Nome AS Disciplina
FROM
    Usuario U
JOIN
    Professor P ON P.ID_Professor = U.ID
JOIN
    Unidade_Escola UE ON UE.ID_Unidade = P.ID_Unidade
JOIN
    Curso C ON C.ID_Unidade = UE.ID_Unidade
JOIN
    Departamento_Academico DA ON DA.Codigo_Unico = C.Codigo_Unico_Departamento
JOIN
    Professor_Ministra_Turma PMT ON PMT.ID_Professor = U.ID
JOIN
    Disciplina D ON D.ID = PMT.ID_Disciplina
WHERE
    PMT.Periodo_Letivo = '2025/1'
    AND DA.Nome = 'Departamento de Ciência da Computação'
ORDER BY
    Nome_Professor;
```

4. Encontrar cursos que não tiveram alunos matriculados no último período letivo.

```sql
-- Encontrar cursos que não tiveram alunos matriculados no último período letivo

SELECT DISTINCT
    C.Codigo_Unico AS codigo_curso,
    C.Nome AS nome_curso
FROM
    Curso C
JOIN
    Disciplina_Compoe_Curso DCC ON DCC.Codigo_Unico_Curso = C.Codigo_Unico
JOIN
    Turma T ON T.ID_Disciplina = DCC.ID_Disciplina
LEFT JOIN
    Matricula_Corresponde_Turma MCT ON MCT.ID_Disciplina = T.ID_Disciplina AND
                                       MCT.Periodo_Letivo = T.Periodo_Letivo
WHERE
    MCT.ID_Matricula IS NULL AND
    T.Periodo_Letivo = '2025/1';
```

5. Listar a avaliação média que cada professor recebeu pelas disciplinas lecionadas no último período letivo.

```sql
-- Consultar a avaliação média de cada professor que lecionou uma disciplina no último período

SELECT
    U.ID,
    U.Nome,
    U.Sobrenome,
    ROUND(AVG(A.Classificacao_Didatica), 2) AS Media_Didatica,
    ROUND(AVG(A.Classificacao_Material_Apoio), 2) AS Media_Material_Apoio,
    ROUND(AVG(A.Classificacao_Conteudo), 2) AS Media_Conteudo
FROM
    Avaliacao A
JOIN
    Professor_Ministra_Turma PMT
    ON PMT.ID_Disciplina = A.ID_Disciplina
    AND PMT.Periodo_Letivo = A.Periodo_Letivo
JOIN
    Usuario U ON U.ID = PMT.ID_Professor
WHERE
    A.Periodo_Letivo = '2024/2'
GROUP BY
    U.ID, U.Nome, U.Sobrenome
ORDER BY
    U.ID;
```

6. Calcular a receita gerada por cada curso com base nos alunos inscritos e suas bolsas, taxas e descontos.

```sql
-- Calcula a receita gerada por cada curso com base nos alunos matriculados

SELECT
    C.Codigo_Unico AS Codigo_Curso,
    C.Nome AS Nome_Curso,
    ROUND(SUM(M.Taxa * (1 - (COALESCE(M.Bolsa, 0) + COALESCE(M.Desconto, 0)) / 100.0)), 2) AS Receita_Liquida
FROM
    Curso C
JOIN
    Disciplina_Compoe_Curso DCC ON DCC.Codigo_Unico_Curso = C.Codigo_Unico
JOIN
    Matricula_Corresponde_Turma MCT ON MCT.ID_Disciplina = DCC.ID_Disciplina
JOIN
    Matricula M ON M.ID_Matricula = MCT.ID_Matricula
WHERE
    M.Status = 'Ativa' AND
    MCT.Periodo_Letivo = '2024/2'
GROUP BY
    C.Codigo_Unico, C.Nome
ORDER BY
    Receita_Liquida DESC;
```

7. Listar os alunos matriculados em cada turma do semestre atual

```sql
-- Lista os Alunos matriculados em cada turma do semestre atual

SELECT
    U.ID AS ID_Aluno,
    U.Nome AS Nome_Aluno,
    U.Sobrenome AS Sobrenome_Aluno,
    T.ID_Disciplina AS ID_Disciplina,
    T.Periodo_Letivo AS Periodo_Disciplina
FROM
    Turma T
JOIN
    Matricula_Corresponde_Turma MCT ON MCT.ID_Disciplina = T.ID_Disciplina AND
                                       MCT.Periodo_Letivo = T.Periodo_Letivo
JOIN
    Matricula M ON M.ID_Matricula = MCT.ID_Matricula
JOIN
    Usuario U ON U.ID = M.ID_Aluno

WHERE
    T.Periodo_Letivo = '2025/1'

ORDER BY
    T.ID_Disciplina;
```

1.7 Índices

a) Índice GIN

O primeiro índice que optamos por criar é o índice de “full-text index”, que serve para acelerar a busca de texto dentro da coluna “Comentario” na tabela Avaliacao. Esse índice pode ser de grande utilidade, pois pode acelerar a busca por palavras-chave presentes nas diversas avaliações feitas por alunos. Por exemplo, caso se queira saber como está sendo a opinião dos alunos sobre a didática das aulas, basta buscar o termo “didática”. Outro exemplo de uso é quando se quer achar comentários positivos, então se busca pelo termo “ótimo”.

Dessa maneira, optamos por utilizar o tipo “GIN (Generalized Inverted Index)”, pois é ideal para dados textuais e busca por palavras.

Abaixo, estão os comandos que utilizamos para realizar a comparação da consulta com e sem o índice:

I) Com índice

```sql
CREATE INDEX idx_avaliacao_comentario_gin ON Avaliacao USING
gin(to_tsvector('portuguese', Comentario));
```

```sql
EXPLAIN ANALYZE
SELECT *
FROM Avaliacao
WHERE to_tsvector('portuguese', Comentario) @@ to_tsquery('portuguese',
'ótimo');
```

	QUERY PLAN text
1	Bitmap Heap Scan on avaliacao (cost=8.54..12.81 rows=1 width=98) (actual time=0.022..0.026 rows=8 loops=1)
2	Recheck Cond: (to_tsvector('portuguese'::regconfig, (comentario)::text) @@ "ótim"::tsquery)
3	Heap Blocks: exact=2
4	-> Bitmap Index Scan on idx_avaliacao_comentario_gin (cost=0.00..8.54 rows=1 width=0) (actual time=0.013..0.013 rows=8 loop...
5	Index Cond: (to_tsvector('portuguese'::regconfig, (comentario)::text) @@ "ótim"::tsquery)
6	Planning Time: 0.249 ms
7	Execution Time: 0.048 ms

Tempo de execução com índice: 0.048ms

II) Sem índice

```
DROP INDEX IF EXISTS idx_avaliacao_comentario_gin;
```

```
EXPLAIN ANALYZE
SELECT *
FROM Avaliacao
WHERE to_tsvector('portuguese', Comentario) @@ to_tsquery('portuguese',
'ótimo');
```

	QUERY PLAN text
1	Seq Scan on avaliacao (cost=0.00..17.75 rows=1 width=98) (actual time=0.783..4.276 rows=8 loops=...
2	Filter: (to_tsvector('portuguese'::regconfig, (comentario)::text) @@ "ótim"::tsquery)
3	Rows Removed by Filter: 52
4	Planning Time: 0.408 ms
5	Execution Time: 4.323 ms

Tempo de execução sem índice: 4.323ms

b) Índice HASH

Outro índice que criamos foi o do tipo “HASH”, com o intuito de realizar consultas otimizadas de igualdade simples, marcadas pelo operador “ = ”. Esse tipo de índice tem uma ótima performance em operações de igualdade, sendo esse o motivo que ele foi utilizado para pesquisar períodos letivos específicos na tabela “Turma”.

No contexto de um sistema para gerenciar atividades acadêmicas e administrativas, procurar os dados de turmas correspondentes a um determinado período letivo se mostra de grande importância. Como se trata de uma instituição de ensino, com o passar do tempo a tendência do número de turmas é só aumentar, logo uma consulta otimizada pelo índice tem uma expressiva utilidade.

I) Com índice

```
CREATE INDEX idx_turma_periodo_hash ON Turma USING hash
(Periodo_Letivo);
```

```
EXPLAIN ANALYZE
SELECT * FROM Turma WHERE Periodo_Letivo = '2025/1';
```

	QUERY PLAN text
1	Bitmap Heap Scan on turma (cost=4.12..7.32 rows=16 width=21) (actual time=0.017..0.021 rows=16 loops=1)
2	Recheck Cond: ((periodo_letivo)::text = '2025/1'::text)
3	Heap Blocks: exact=1
4	-> Bitmap Index Scan on idx_turma_periodo_hash (cost=0.00..4.12 rows=16 width=0) (actual time=0.007..0.007 rows=16 loop...
5	Index Cond: ((periodo_letivo)::text = '2025/1'::text)
6	Planning Time: 0.093 ms
7	Execution Time: 0.040 ms

Tempo de execução com índice: 0.040ms

II) Sem índice

```
DROP INDEX IF EXISTS idx_turma_periodo_hash;
```

```
EXPLAIN ANALYZE
SELECT * FROM Turma WHERE Periodo_Letivo = '2025/1';
```

	QUERY PLAN text
1	Seq Scan on turma (cost=0.00..8.23 rows=16 width=21) (actual time=0.051..0.340 rows=16 loops=...
2	Filter: ((periodo_letivo)::text = '2025/1'::text)
3	Rows Removed by Filter: 402
4	Planning Time: 0.221 ms
5	Execution Time: 0.385 ms

Tempo de execução sem índice: 0.385ms

c) Índice B-tree (padrão)

Finalmente, o último índice que optamos utilizar foi no atributo “Nome” da tabela “Curso”. Naturalmente, em um sistema de uma instituição de ensino, os alunos terão o interesse de realizar consultas com base no nome dos cursos oferecidos. Desse modo, acelerar as consultas com base neste atributo se mostra interessante, pois é um atributo que é requerido com bastante frequência nas buscas.

Pensando nisso, criamos esse índice com o tipo padrão, o “B-tree”, pela sua versatilidade de eficiência em comparações de igualdade (=), com intervalos (<, >, BETWEEN) e com ordenação (ORDER BY).

Obs: Nessa consulta, foi utilizado o comando “SET enable_seqscan = OFF”, para possibilitar a comparação de performance entre a consulta sem o índice e com o índice. Por padrão, mesmo com o índice criado, é o SGBD que escolhe se a consulta será realizada com o índice, e ele estava optando por não usá-lo. Por esse motivo, utilizamos esse comando para forçar o uso do índice e ser possível observar a diferença de performance.

I) Com índice

```
CREATE INDEX idx_curso_nome ON Curso(Nome);
```

```
SET enable_seqscan = OFF;
```

```
EXPLAIN ANALYZE
SELECT Nome FROM Curso ORDER BY Nome;
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	QUERY PLAN text
1	Index Only Scan using idx_curso_nome on curso (cost=0.27..53.05 rows=331 width=22) (actual time=0.020..0.182 rows=331 loops=...
2	Heap Fetches: 331
3	Planning Time: 0.055 ms
4	Execution Time: 0.206 ms

Tempo de execução com índice: 0.206ms

II) Sem índice

```
DROP INDEX IF EXISTS idx_curso_nome;
```

```
SET enable_seqscan = ON;
```

```
EXPLAIN ANALYZE
SELECT Nome FROM Curso ORDER BY Nome;
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1

	QUERY PLAN text
2	Sort Key: nome
3	Sort Method: quicksort Memory: 25kB
4	-> Seq Scan on curso (cost=0.00..11.31 rows=331 width=22) (actual time=0.041..0.280 rows=331 loop...
5	Planning Time: 0.230 ms
6	Execution Time: 2.332 ms

Tempo de execução sem índice: 2.332ms

1.8 Índices

a) Visão 1

```sql
CREATE VIEW view_PreRequisitos AS
-- pré requisito é uma disciplina
SELECT
    C.Nome AS curso,
    D.Nome AS pre_requisito,
    'Disciplina' AS tipo_pre_requisito
FROM
    Curso AS C
JOIN
    Curso_PreRequisita_Disciplina CPD ON CPD.Codigo_Unico_Curso = C.Codigo_Unico
JOIN
    Disciplina D ON D.ID = CPD.ID_Disciplina_PreRequisito

UNION

-- pré requisito é um curso
SELECT
    C1.Nome AS curso,
    C2.Nome AS pre_requisito,
    'Curso' AS tipo_pre_requisito
FROM
    Curso AS C1
JOIN
    Curso_PreRequisita_Curso CPC ON CPC.Codigo_Unico_Curso = C1.Codigo_Unico
JOIN
    Curso AS C2 ON C2.Codigo_Unico = CPC.Codigo_Unico_Curso_PreRequisito;
```

A visão acima realiza uma consulta de todos os pré-requisitos para os cursos(que possuem pré-requisitos). Eles são divididos em 2 categorias, curso e disciplina, que estão dispostos na coluna “tipo_pre_requisito”.

Analisando a consulta em questão, ela é relativamente complexa, portanto a criação de uma visão é justificada. Além disso, dado o contexto de um sistema de instituição de ensino, saber os pré requisitos de um curso é de extrema importância para os alunos, para saberem se eles podem ou não realizar determinado curso. Logo, é uma consulta que seria utilizada diversas vezes.

b) Visão 2

```sql
CREATE VIEW view_Avaliacoes_Insatisfatorias AS
SELECT
    U.Nome || ' ' || U.Sobrenome AS nome_completo,
    T.ID_Disciplina AS disciplina,
    T.Periodo_Letivo AS periodo_letivo,
    ROUND(
        (AVA.Classificacao_Didatica + AVA.Classificacao_Material_Apoio +
        AVA.Classificacao_Conteudo + AVA.Classificacao_Infraestrutura) / 4.0, 2) AS media_das_avaliacoes,
    AVA.Comentario AS comentario
FROM
    Avaliacao AVA
JOIN
    Aluno ALU ON ALU.ID_Aluno = AVA.ID_Aluno
JOIN
    Usuario U ON U.ID = ALU.ID_Aluno
JOIN
    Turma T ON (T.ID_Disciplina, T.Periodo_Letivo) = (AVA.ID_Disciplina, AVA.Periodo_Letivo)
WHERE
    ROUND(
        (AVA.Classificacao_Didatica + AVA.Classificacao_Material_Apoio +
        AVA.Classificacao_Conteudo + AVA.Classificacao_Infraestrutura) / 4.0, 2) < 3.5;
```

A visão acima possui uma consulta de todas as avaliações(comentários) cuja média de classificação da didática, material de apoio, conteúdo e infraestrutura a uma disciplina em um determinado período letivo são menores que 3.5 pontos.

Optamos por realizar uma visão com essa consulta, pois entendemos que consultar as avaliações negativas realizadas por alunos seria importante para a administração, com o intuito de melhorar o ensino e ouvir as demandas dos alunos. Já que envolve alguns cálculos e alguns joins, a visão torna mais simples a realização da consulta.

c) Visão 3

```sql
CREATE VIEW view_Docentes_Disciplinas AS
SELECT
    U.Nome || ' ' || U.Sobrenome AS nome_docente,
    AE.Area AS area_especializacao_docente,
    TIT.Titulo AS titulacao_docente,
    D.Nome AS disciplina_ministrada,
    TUR.Periodo_Letivo AS periodo_letivo
FROM
    Usuario U
JOIN
    Professor P ON U.ID = P.ID_Professor
JOIN
    Area_Especializacao AE ON AE.ID_Professor = P.ID_Professor
JOIN
    Titulacao TIT ON TIT.ID_Professor = P.ID_Professor
JOIN
    Professor_Ministra_Turma PMT ON PMT.ID_Professor = P.ID_Professor
JOIN
    Disciplina D ON D.ID = PMT.ID_Disciplina
JOIN
    Turma TUR ON (TUR.ID_Disciplina, TUR.Periodo_Letivo) = (PMT.ID_Disciplina, PMT.Periodo_Letivo);
```

A visão a seguir realiza uma consulta que tem múltiplas utilidades. Esse comando busca os professores da instituição e mostram as suas áreas de

especialização, títulos e as disciplinas ministradas dado um período letivo. Por exemplo, caso um aluno esteja em dúvida de qual professor ele gostaria de fazer uma iniciação científica, ele poderia ver a tabela e, de acordo com as especializações, títulos e disciplinas ministradas, escolher o professor que mais se adequasse às suas necessidades. Outro exemplo, é na hora de escolher uma turma, e necessita-se escolher entre dois professores que ministram essa determinada disciplina. Dependendo da especialização ou título que o docente possui, os alunos escolheriam um professor em detrimento do outro.

A consulta acima envolve 6 JOINS, uma quantidade expressiva para ser escrita toda vez que fosse necessário realizar a consulta. Dessa maneira, a visão salva essa consulta de forma simples.

1.9 Vídeo

Neste exercício, foi gravado um vídeo acerca do funcionamento das consultas criadas anteriormente. Este vídeo pode ser acessado pelo seguinte link: https://www.youtube.com/watch?v=Ea6fa_h3MQ4.

1.10 Reflexão

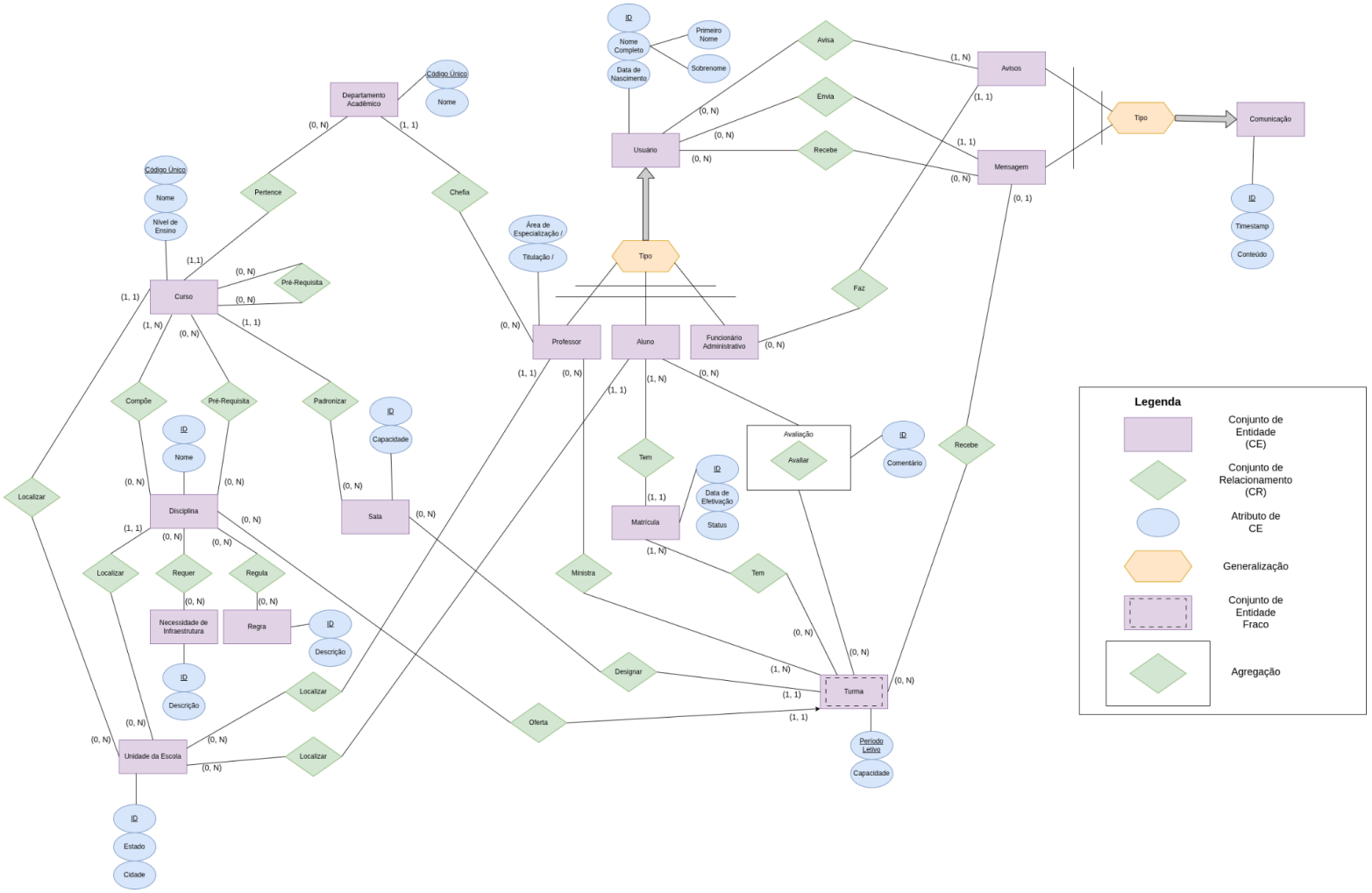
Durante a construção do trabalho, cada membro teve uma percepção diferente acerca das dificuldades e aprendizados, pois a equipe optou por dividir os exercícios igualmente entre os membros.

Na etapa da confecção de consultas SQL, sentiu-se uma dificuldade em saber de qual relação partir, por quais relações passar e em qual relação chegar, ao utilizar o comando SELECT em uma consulta mais complexa. À medida que a complexidade do sistema cresce(maior número de relações e relacionamentos entre essas), fica cada vez mais difícil realizar tal tarefa.

Outro ponto de dificuldade encontrado por nós foi a necessidade de aprender a utilizar a ferramenta pgAdmin 4 do zero, sem ter nenhum contato prévio antes da realização dessa matéria. O processo de aprendizado, principalmente no início, não foi tão trivial. No entanto, com o decorrer do tempo, fomos nos acostumando, e agora dominamos mais a ferramenta, o que agregou bastante no nosso conhecimento.

Por fim, um último ponto de dificuldade presente foi adicionar dados suficientes para se notar a diferença de performance ocasionada pelos índices. Como são dados criados artificialmente, inventá-los foi trabalhoso, ainda mais considerando que certos elementos que atuavam como Chave Estrangeira deveriam existir na tabela referenciada. Dessa maneira, para que a inserção dos dados fosse válida, ela tinha que respeitar minuciosamente as restrições das chaves estrangeiras, o que ocasionou em um trabalho manual a mais.

Em geral, a execução desse projeto foi bem proveitosa ao grupo, visto que foi possível aplicar na prática os conhecimentos adquiridos nas aulas, modelando diagramas, mapeando-os, normalizando-os, etc. No entanto,um aprendizado teve um destaque, na opinião do nosso grupo: a capacidade de transformar uma ideia

abstrata, que no início foi dada apenas como um texto corrido, em um banco de dados com relações, relacionamentos e dados concretos, mais tangíveis. Foi possível aprender e praticar esse processo por inteiro no trabalho, o que conferiu uma considerável bagagem ao nosso conhecimento.

2. Diagrama Entidade-Relacionamento